Hierarchical Transformer Preconditioning for Interactive Physics Simulation

ANONYMOUS AUTHOR(S)
SUBMISSION ID: 0000

Neural preconditioners promise data-driven priors for the linear systems that dominate real-time physics simulation, but existing graph-neural designs struggle with long-range couplings: their nonzero pattern is constrained to that of the system matrix, and information between distant degrees of freedom must traverse many message-passing layers. We introduce a neural preconditioner anchored to a fixed weak-admissibility $\mathcal{H}$-matrix partition. The partition is computed analytically and reused for all inputs, giving the network a strong multiscale structural bias: a transformer stack handles dense diagonal blocks at full rank, and a second stack emits low-rank factors for off-diagonal tiles at a single coarse resolution regardless of tile size. Because every off-diagonal block is processed at the same token count, the architecture's compute and memory scale linearly in problem size, $\Theta(N)$, rather than the $O(N \log N)$ of classical $\mathcal{H}$-matrix arithmetic. We train the preconditioner self-supervised with a cosine-similarity Hutchinson probe loss that, unlike Frobenius-style objectives, is invariant to both the scale and the absolute location of the eigenvalues of MA, letting the network cluster the spectrum wherever the residual makes alignment easiest. At inference, the entire PCG step (preconditioner application plus sparse matvec) is a sequence of batched GEMMs over contiguous memory, with no triangular solves or data-dependent branching, captured end-to-end in a single CUDA Graph. On a stiff multiphase pressure-Poisson benchmark with 8 192 unknowns, our preconditioner reduces the PCG iteration count from 2 163 (unpreconditioned, GPU) to 141 and produces an end-to-end solve time of 15.7 ms (4.0 ms inference + 11.7 ms PCG), over 3× faster than GPU Jacobi and over 18× faster than the next-best classical GPU preconditioner we evaluate.

CCS Concepts: • **Computing methodologies** → **Physical simulation**; *Neural networks.*

Additional Key Words and Phrases: preconditioning, iterative solvers, hierarchical matrices, transformers, neural operators, physics simulation, real-time graphics

ACM Reference Format:
Anonymous Author(s). 2026. Hierarchical Transformer Preconditioning for Interactive Physics Simulation. *ACM Trans. Graph.* 45, 6, Article 1 (December 2026), 8 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Real-time simulation of fluids, soft bodies, and coupled multiphysics systems requires solving a large sparse linear system $A\mathbf{x} = \mathbf{b}$ at every time step, almost always with preconditioned conjugate gradient (PCG). The preconditioner $M \approx A^{-1}$ is the dominant design variable: it must be cheap to apply and must cluster the eigenvalues of MA tightly enough to drive PCG to its tolerance in a few dozen iterations. In an interactive budget of a few milliseconds per frame, classical preconditioners that re-amortize a heavy setup at every frame—incomplete Cholesky (IC) and algebraic multigrid (AMG) above all—become impractical, both because their setup costs are unpredictable and because their application is dominated by sequential triangular solves or level-by-level synchronization that map poorly onto modern GPUs [Naumov et al. 2015; Yang et al. 2025].

Machine learning offers an appealing alternative: given a distribution of simulation states, a neural preconditioner can be trained once and amortized across frames at near-constant, predictable inference cost. Graph neural networks (GNNs) have become the dominant architecture for this purpose [Chen 2024; Häusner et al. 2023; Li et al. 2023; Yang et al. 2025], because the natural graph interpretation of a sparse matrix lets message-passing layers propagate information through its nonzero pattern. However, this same choice imposes two fundamental limitations that become acute in real-time simulation. First, GNN preconditioners typically inherit the sparsity pattern of A or of an incomplete factor of A: non-adjacent interactions are either ignored entirely or only approximated through repeated graph convolutions, which require many layers to propagate long-range information and tend to oversmooth local detail along the way [Trifonov et al. 2025; Wu et al. 2019]. Second, the dominant operations at apply time—triangular solves on a learned IC factor, or sparse matrix-vector products on a learned SPAI—are bandwidth-bound and either serialize on data hazards or scatter through irregular memory access [Yang et al. 2025].

We propose a different architecture that targets both limitations simultaneously. Our starting observation is structural: simulation codes routinely reorder degrees of freedom along a space-filling curve (Morton/Z) or a bandwidth-reducing permutation (reverse Cuthill–McKee), so under such orderings near-diagonal entries of A describe spatially local interactions while far-diagonal entries describe distant ones. This is the $\mathcal{H}$-matrix-compatible banded structure familiar from hierarchical matrices [Hackbusch 1999; Hackbusch and Khoromskij 2002]: well-separated index ranges admit low-rank approximation, and the rank decreases with separation. The same hierarchy underlies multigrid analysis: high-frequency residual components are local and require dense corrections, while low-frequency components—bulk pressure shifts, large-scale stress modes—are nonlocal and require correspondingly nonlocal corrections [Briggs et al. 2000]. The upshot is that a banded A already encodes the spectral hierarchy its inverse needs to respect.

Approach. We anchor the preconditioner to a weak-admissibility $\mathcal{H}$-matrix partition [Hackbusch and Khoromskij 2002], computed analytically and held fixed throughout training and inference. With minimum block size L and $K = N/L$ leaves, the partition consists of K dense diagonal blocks of size $L \times L$ (one per leaf) and a static set of off-diagonal tiles, each spanning $S \times S$ leaves with $S = 2^j$, indexed by leaf coordinates. A two-stream transformer (LeafOnlyNet) operates on this layout: a *diagonal* stream emits one factor matrix per leaf, and an *off-diagonal* stream emits one factor matrix per tile. Crucially, every off-diagonal tile is processed at the *same* coarse token count $L_s = L/p$, regardless of S; the implied full-resolution

2026. ACM 1557-7368/2026/12-ART1
https://doi.org/10.1145/nnnnnnn.nnnnnnn

block then has rank at most L_s on a $SL \times SL$ region, so the rank fraction $L_s/(SL)$ decreases automatically as S grows—the structural bias appropriate for $\mathcal{H}$-matrix compression. To break the locality of within-block attention without giving up the $\Theta(N)$ budget, we route information between layers through *highway* buffers organized as axial (row/column-band) buffers and a single global summary token (Section 4.4).

The resulting system addresses each of the limitations above. (i) *Full-graph coverage*: long-range couplings are represented explicitly by the off-diagonal stream rather than filtered through repeated message passing on a fixed sparsity pattern. (ii) *No triangular solves*: preconditioner application is a fixed sequence of batched matrix multiplications on contiguous tensors. (iii) *Predictable cost*: the partition is static, so FLOP count and memory traffic are data-independent and the entire PCG inner loop (preconditioner apply + sparse matvec + scalar updates) captures cleanly into a single CUDA Graph. (iv) *Linear scaling*: both forward inference and per-iteration application are $\Theta(N)$, with leading constants set by hyperparameters L, L_s, d, n_l rather than by problem-dependent fill-in.

Contributions. (1) A neural preconditioner anchored to a fixed weak-admissibility $\mathcal{H}$-matrix partition, with a uniform-resolution off-diagonal stream that yields $\Theta(N)$ inference and apply, unlike classical $\mathcal{H}$-matrix arithmetic. (2) Multiscale highway connections that propagate axial and global context between transformer layers without breaking the linear budget. (3) A scale- and location-invariant cosine Hutchinson loss that, unlike Frobenius- or SAI-style objectives [Yang et al. 2025], is invariant to both the scale and the absolute location of the eigenvalues of MA—a property that matters for stiff, multiphase systems whose spectra do not naturally cluster around 1. (4) An end-to-end allocation-free apply path that captures into a single CUDA Graph, giving zero CPU dispatch overhead per PCG iteration.

2 Related Work

Classical iterative solvers. Preconditioning sparse SPD systems is a textbook concern of scientific computing [Saad 2003]. Widely used algebraic preconditioners include Jacobi, incomplete Cholesky/LU (IC/ILU) [Lin and Moré 1999], sparse approximate inverses (SPAI) [Benzi et al. 1996; Kolotilina and Yeremin 1993], and algebraic multigrid (AMG) [Ruge and Stüben 1987]. Each trades approximation quality against arithmetic and memory cost. IC achieves strong spectral approximation but requires triangular solves whose data hazards resist GPU parallelization [Liu et al. 2016; Yamazaki et al. 2020]; SPAI avoids these solves but its construction is a pattern-by-pattern least-squares optimization that often yields suboptimal condition numbers [Yang et al. 2025]; AMG offers excellent convergence on elliptic problems but requires sensitive setup and irregular V-cycle communication [Naumov et al. 2015]. In an interactive setting, setup cost alone often exceeds the per-frame budget; our amortized neural construction inverts this trade-off.

Neural operators and learned PDE solvers. Neural operators including Fourier-based [Li et al. 2021], DeepONet-based [Lu et al. 2021], and graph-based [Li et al. 2020a,b] variants learn function-space mappings and have been used both as PDE surrogates and as preconditioners inside flexible Krylov solvers [Rudikov et al. 2024]. Physics-informed networks [Raissi et al. 2019] embed PDE residuals directly into the loss. Our approach is complementary and purely algebraic: we operate on the assembled sparse matrix and simulation-state node features, with no reference to an underlying PDE or coordinate system beyond the (already-required) spatial node positions, so the same model applies to non-uniform materials and irregular discretizations.

Learning-based preconditioners. A line of work trains GNNs to predict the nonzeros of an incomplete factor of A [Häusner et al. 2023; Li et al. 2023; Trifonov and Güttel 2024], replacing classical fill-in heuristics with a learned forward pass but inheriting the triangular-solve bottleneck at apply time and requiring local message passing to capture a global elimination tree [Trifonov et al. 2025]. Yang *et al.* [Yang et al. 2025] sidestep the apply-time issue by having a GNN emit a sparse $GG^\top$ approximate inverse evaluated as two SpMVs; their scale-invariant Frobenius ("SAI") loss enables training without solution vectors. Chen [Chen 2024] reports strong results across the SuiteSparse collection on ill-conditioned problems where classical IC and AMG fail. We extend the SPAI direction along a different axis: rather than restricting the preconditioner to the sparsity pattern of A, we structure it as an $\mathcal{H}$-matrix that explicitly represents long-range couplings, trade SpMV bandwidth-bound apply for compute-bound batched GEMM, and replace the SAI loss with a cosine objective that drops the implicit eigenvalue-location prior.

Hierarchical neural architectures for PDEs. Multiscale neural networks [Fan et al. 2019], multipole graph neural operators [Li et al. 2020a], neural-FMM Green's-function networks [Fognini et al. 2025], and neural-HSS solvers [Sittoni et al. 2026] embed FMM- or $\mathcal{H}$-matrix-style hierarchical structure into neural networks, but as forward surrogates that *replace* the numerical solve. Closer to our setting, neural approximate inverse preconditioners [Xu et al. 2025] and multigrid-inspired neural iterative solvers [Luz et al. 2020; Lyu et al. 2026] learn operators that accelerate an outer iteration. Compared with these, we differ in three ways: (i) we use a static weak-admissibility $\mathcal{H}$-matrix partition rather than a multigrid hierarchy or a learned partition, (ii) the apply path is allocation-free and captures into a single CUDA Graph, and (iii) the cosine Hutchinson training objective, which to our knowledge has not appeared in the prior literature on learned preconditioners.

Hierarchical matrix techniques. Classical $\mathcal{H}$-matrix and $\mathcal{H}^2$-matrix arithmetic exploits low-rank structure in well-separated sub-blocks [Börm et al. 2003; Hackbusch 1999; Hackbusch and Khoromskij 2002]; HODLR variants [Hartland et al. 2023] apply the same structural prior to Hessians in inverse problems. We retain the partition and the weak admissibility framework, but replace classical analytical low-rank approximations of fixed off-diagonal blocks with transformer-generated factors at a uniform coarse rank—letting the network discover, from simulation-state features, which entries of the inverse the local low-rank approximation should resolve.

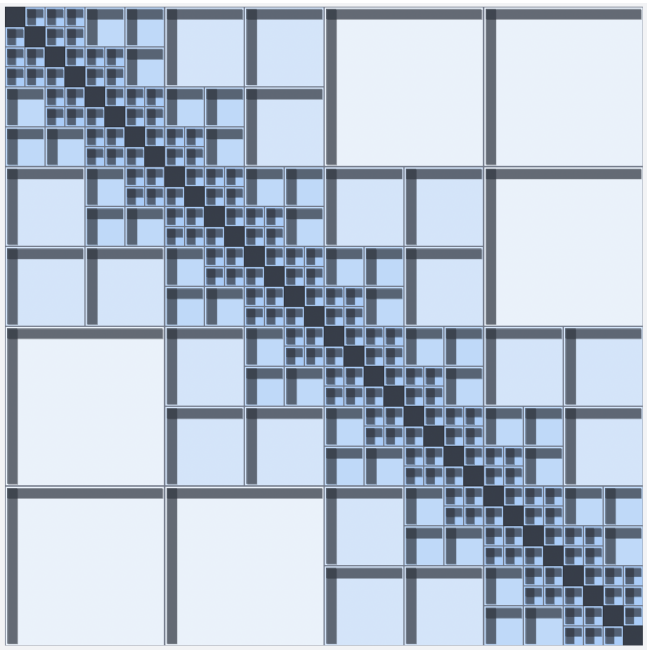

Fig. 1. Weak-admissibility $\mathcal{H}$-matrix partition (illustrative leaf count). Dense diagonal blocks lie along the main diagonal; admissible off-diagonal tiles double in size as separation from the diagonal grows. Translucent red overlays the upper-triangle row and column index sets associated with one exemplar tile—the same index sets accumulated into the row and column highway buffers during training (Section 4.4).

3 Background

Preconditioned conjugate gradient. We seek $\mathbf{x} \in \mathbb{R}^N$ satisfying $A\mathbf{x} = \mathbf{b}$ with A SPD. Preconditioned CG introduces $M \approx A^{-1}$ and applies it to the residual at every iteration; convergence rate is governed by the spread of the eigenvalues of MA, scale-free in A. Throughout we treat the preconditioner as a linear (matrix-free) operator and never materialize M.

Weak-admissibility $\mathcal{H}$-matrices. Partition the index set $\{0, \ldots, N-1\}$ into $K = N/L$ contiguous *leaves* of size L. A block coupling leaf ranges $\mathcal{I}, \mathcal{J}$ is *weakly admissible* if $|\mathcal{I}| \le \eta \operatorname{dist}(\mathcal{I}, \mathcal{J})$, where dist is the gap in leaf-index space. An $\mathcal{H}$-matrix retains diagonal blocks at full rank and replaces every weakly admissible off-diagonal block with a low-rank factorization. We compute the partition by assigning to each leaf pair the largest admissible block size of S leaves on a side ($S = 2^j$) and collecting the unique tiles; because A is symmetric we keep only the $r_0 \le c_0$ tiles and apply both halves symmetrically at runtime. For our default $K = 128$ leaves at $\eta = 1$, the partition has $M_{\mathcal{H}} = 487$ unique upper-triangle off-diagonal tiles ($\approx 3.8K$); the count grows nearly linearly with K in our regime, keeping storage and compute at $O(N)$.

Hutchinson trace estimation. The Hutchinson estimator [Hutchinson 1989], $\operatorname{tr}(B) = \mathbb{E}_{\mathbf{z}\sim\mathcal{N}(0,I)}[\mathbf{z}^\top B\mathbf{z}]$, lets us measure preconditioner quality without ever materializing A^{-1}: drawing probes $\mathbf{z}$, computing $A\mathbf{z}$ via SpMV, applying M to obtain $\hat{\mathbf{z}} = M(A\mathbf{z})$, and measuring the alignment of $\hat{\mathbf{z}}$ with $\mathbf{z}$ yields a differentiable surrogate for $MA \approx I$.

4 Method

LeafOnlyNet is a neural network f_θ mapping a simulation state and the sparse matrix A to a packed tensor encoding all factors of an $\mathcal{H}$-matrix approximate inverse. The architecture has four stages: a physics-aware encoder, a diagonal transformer stack over the K leaves, an off-diagonal transformer stack over the $M_{\mathcal{H}}$ admissible tiles, and linear decoder heads. Highway connections optionally couple the two stacks at every layer.

4.1 Physics-Aware Encoder

Each simulation node carries position $\mathbf{p}_i \in \mathbb{R}^3$, velocity $\mathbf{v}_i \in \mathbb{R}^3$, and per-problem features (density, pressure, material parameters, etc.). A two-layer MLP lifts these—together with an optional broadcast global-feature vector encoding simulation-level context—to dimension d. The lifted embeddings then pass through $n_{\text{gcn}} = 2$ residual graph-convolutional layers [Kipf and Welling 2017]: each layer concatenates a self-projection of $\mathbf{h}_i$ with an A-weighted neighbor mean and feeds the result to a small GELU MLP, giving every node an embedding aware of its local matrix entries and the physical state of nearby particles. Crucially, this is the only stage that observes individual graph edges; all subsequent processing operates on the block layout of the $\mathcal{H}$-matrix partition, not the graph itself, which decouples downstream cost from edge count.

4.2 Diagonal Transformer Stack

The diagonal stream produces, for each of the K leaves, an $L_s \times L_s$ factor matrix F_k such that the k-th diagonal block of the approximate inverse is $F_k F_k^\top$. We mean-pool the encoder output within each leaf by factor p_{diag} to a sequence of length $L_s = L/p_{\text{diag}}$ tokens per block, and process the resulting $K\,L_s$ tokens through n_l transformer layers [Vaswani et al. 2017] that perform attention strictly within each leaf. The within-block attention has cost $O(KL_s^2 d) = O(N)$.

Two physics-derived modifications make the attention A-aware. First, attention logits are augmented with a per-head bias produced by a small MLP from the four-dimensional edge descriptor $[\Delta x, \Delta y, \Delta z, A_{ij}]$ for each in-block token pair, so geometry and coupling strength can directly modulate attention. Second, a binary self-marker token flag distinguishes diagonal entries from neighbor entries.

4.3 Off-Diagonal Transformer Stack

For each off-diagonal tile m the off-diagonal stream produces, via two parallel linear heads, a pair of thin factor matrices $U_m, V_m \in \mathbb{R}^{L_s \times L_s}$. Their outer product $B_m = U_m V_m^\top$ is the coarse-resolution off-diagonal contribution: tile m contributes B_m to the rectangular block of the approximate inverse above the diagonal and $B_m^\top$ to the symmetrically located block below, preserving symmetry. We materialize B_m once at the end of the forward pass and store only it in the packed tensor, so the apply path sees one $L_s \times L_s$ matrix per tile rather than a separate U, V pair.

The off-diagonal stream is initialized by row- and column-strip pooling. Static binary weight matrices $W^{\text{row}}, W^{\text{col}} \in \mathbb{R}^{M_{\mathcal{H}} \times K}$ encode, for each tile m, which leaves lie in its row range $[r_0, r_0 + S)$ and column range $[c_0, c_0 + S)$. The strip embedding is

$$\mathbf{s}_m = \left(W^{\text{row}}_{m,:} + W^{\text{col}}_{m,:}\right) \mathbf{H}_{\text{leaves}}, \qquad \mathbf{H}_{\text{leaves}} \in \mathbb{R}^{K \times L \times d}, \tag{1}$$

optionally further pooled along the leaf-token axis by p_{off} so every tile arrives at the off-diagonal stack with the same fixed shape (L_s, d). The stack itself has the same structure as the diagonal one: n_l transformer layers attending over the L_s tokens of each tile, with edge biases derived this time from the mean of $[\Delta x, \Delta y, \Delta z, A_{ij}]$ over all node pairs that share a graph edge across the tile's row and column regions.

The defining property of this stream is that *every off-diagonal tile is processed at the same fixed token count L_s and channel width d, regardless of the tile's physical size SL*. Two consequences follow. First, total off-diagonal cost is $O(M_{\mathcal{H}} L_s^2 d) = O(N d)$, not $O(N \log N)$ as in classical $\mathcal{H}$-matrix arithmetic where rank is allowed to grow with block size. Second, the implied full-resolution block has rank at most L_s on a region of size $SL \times SL$, so the rank fraction $L_s/(SL)$ shrinks automatically as S grows—exactly the prior that makes $\mathcal{H}$-matrix compression effective: distant tiles, encoding smoother low-frequency couplings, are compressed more aggressively than near tiles.

4.4 Highway Connections

A transformer that attends only within its block builds rich within-block representations but cannot directly observe the global state. To restore this, we optionally couple the two stacks through three highway buffers per layer: row and column *axial* buffers $\mathbf{r}_{\text{hw}}, \mathbf{c}_{\text{hw}} \in \mathbb{R}^{B \times N \times d}$, and a single global token $\mathbf{g}_{\text{hw}} \in \mathbb{R}^{B \times d}$.

After the attention sublayer of each transformer block, the current block-token embeddings are scatter-added into their corresponding rows of the axial buffers (off-diagonal block tokens are first repeat-interleaved back to full leaf resolution so each off-diagonal tile contributes uniformly across its S-leaf row and column ranges), and the global token is updated as the mean over all active tokens. Before the FFN sublayer, every token $\mathbf{z}$ at index i reads its highway context and forms the FFN input

$$\tilde{\mathbf{z}} = [\, \mathbf{z} \,;\, \mathbf{r}_{\text{hw}}[i] \,;\, \mathbf{c}_{\text{hw}}[i] \,;\, \mathbf{g}_{\text{hw}} \,] \in \mathbb{R}^{4d}, \tag{2}$$

which is consumed by the FFN's input projection (width $4d$). The highway adds one $O(Nd)$ scatter-gather per layer, preserving overall $\Theta(N)$ scaling.

The four channels participating in each layer—intra-block attention (2D, local), row and column highways (1D, axial), and global token (0D, broadcast)—compose a multiscale communication structure that mirrors the 2D/1D/0D band-row-column-global geometry of the inverse of a banded matrix. We ablate this channel structure in Section 7.4.

4.5 Decoder and Output

After both transformer stacks, three lightweight linear heads project token embeddings to preconditioner factors. A two-layer *leaf head* maps each diagonal-stream token to $\mathbb{R}^{L_s}$, giving one $F_k \in \mathbb{R}^{L_s \times L_s}$ per leaf. Two *off-diag heads* map each off-diagonal-stream token to $\mathbb{R}^{L_s}$, giving $U_m, V_m \in \mathbb{R}^{L_s \times L_s}$ per tile. To extend the off-diagonal contribution back to full node resolution, two *node heads* project per-node diagonal-stream embeddings (upsampled if $p_{\text{diag}} > 1$) to $\mathbb{R}^{L_s}$, giving node-factor matrices $\tilde{U}_k, \tilde{V}_k \in \mathbb{R}^{L \times L_s}$ per leaf. The application section below shows how $\tilde{U}, \tilde{V}, U, V$ together implement an FMM-style restriction-coarse-prolongation chain.

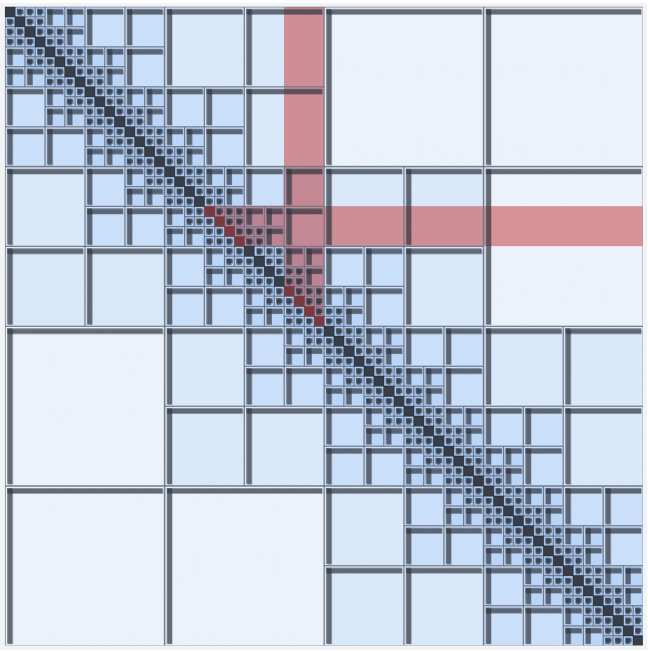

Fig. 2. One off-diagonal tile contributes scatter-add updates to its row-strip and column-strip ranges in the axial highway buffers (translucent red), and to a single broadcast global token. Together with within-block attention, this gives the architecture four communication channels of dimensions 2D / 1D / 1D / 0D per layer.

A scalar gate finally produces a per-node correction λ_i added as $\text{diag}(\boldsymbol{\lambda})\,\text{diag}(A)^{-1}$. Early in training, when the hierarchical factors approximate A^{-1} poorly, this Jacobi residual provides a cheap gradient sink that prevents loss stagnation; as the hierarchical factors improve, the gate's contribution is reduced and capacity shifts to the structured branch. We ablate this component (Section 7.4) and find that removing it both slows training and modestly degrades final iteration count.

All factors are concatenated into a single packed tensor of width $P = KL_s^2 + M_{\mathcal{H}} L_s^2 + 2NL_s + N$, which is what the apply routine consumes.

4.6 Complexity

Let L be the leaf size, $K = N/L$, $L_s = L/p$, and d, n_l be the model width and depth. Encoder cost is $O(Nd^2)$ for bounded-degree graphs. The diagonal stack costs $O(n_l K L_s^2 d) = O(N)$ in attention and $O(n_l K L_s d^2) = O(N)$ in FFN. The off-diagonal stack has $M_{\mathcal{H}} = O(K)$ tiles each of size L_s, so its cost is also $O(N)$. Highway scatter-gather adds $O(Nd)$ per layer. The total inference budget is therefore $\Theta(N)$, with constants set entirely by the fixed hyperparameters d, L, L_s, n_l.

5 Preconditioner Application

PCG calls M on the residual once per iteration. Two design choices keep this hot path allocation-free, branch-free, and dispatchable as a single CUDA Graph. First, M is never formed explicitly: at full resolution its off-diagonal blocks reach $\frac{N}{2}\times\frac{N}{2}$, and assembling AM to recast as left-preconditioned would further densify the off-diagonal couplings and destroy the factored low-rank structure that gives our method its $\Theta(N)$ scaling. Instead we maintain M in the factored form $\{F_k, \tilde{U}_k, \tilde{V}_k\}_k \cup \{B_m\}_m$ and right-precondition. Second, every intermediate buffer is preallocated at solve setup so the PCG inner loop never allocates.

Diagonal contribution. The diagonal blocks are extracted as a strided view (no copy) into the packed tensor. The input residual is reshaped into K leaf segments, optionally mean-pooled to L_s tokens, and a single batched GEMM evaluates $\mathbf{y}_k^{\text{diag}} = F_k\,\mathbf{r}_k$ across all leaves at once; the result is then expanded back to length L if pooled. Total cost is $2KL_s^2 = 2NL_s/L$ FLOPs over K contiguous matrices, with arithmetic intensity $\sim L_s/2$ FLOPs/byte—two orders of magnitude above the scalar Jacobi multiply, and well inside the compute-bound regime on Tensor Core hardware.

Off-diagonal contribution: FMM-style four-stage chain. The node factors $\tilde{U}, \tilde{V}$ project node-resolution data to coarse moments; the coarse blocks $B_m = U_m V_m^\top$ couple moments across well-separated leaf ranges; and the projection is inverted at the end. Concretely, with $\mathbf{r}_k$ the residual on leaf k, $\mathcal{R}_m = [r_0^{(m)}, r_0^{(m)} + S^{(m)})$ and $\mathcal{C}_m = [c_0^{(m)}, c_0^{(m)} + S^{(m)})$:

$$\hat{\mathbf{u}}_k = \tilde{U}_k^\top \mathbf{r}_k, \qquad \hat{\mathbf{v}}_k = \tilde{V}_k^\top \mathbf{r}_k \qquad \text{(restriction)} \tag{3}$$

$$\mathbf{s}_m^{\mathrm{r}} = \sum_{k\in\mathcal{R}_m} \hat{\mathbf{u}}_k, \qquad \mathbf{s}_m^{\mathrm{c}} = \sum_{k\in\mathcal{C}_m} \hat{\mathbf{v}}_k \qquad \text{(strip aggr.)} \tag{4}$$

$$\mathbf{t}_m^{\mathrm{c}} = B_m^\top \mathbf{s}_m^{\mathrm{r}}, \qquad \mathbf{t}_m^{\mathrm{r}} = B_m\,\mathbf{s}_m^{\mathrm{c}} \qquad \text{(coarse coupling)} \tag{5}$$

$$\Delta\mathbf{y}_k = \tilde{U}_k \textstyle\sum_{m:k\in\mathcal{R}_m} \mathbf{t}_m^{\mathrm{r}} + \tilde{V}_k \sum_{m:k\in\mathcal{C}_m} \mathbf{t}_m^{\mathrm{c}} \qquad \text{(prolong.)} \tag{6}$$

Stages 1 and 4 are batched GEMMs of shape (K, L, L_s) over contiguous memory; stage 3 is a batched GEMM of shape $(M_{\mathcal{H}}, L_s, L_s)$ that fully occupies Tensor Cores; stage 2 is two scatter-add operations whose gather indices are precomputed once at module init from the static partition. Total cost is $O(NL_s)$. Symmetry of the preconditioner means we store only the $r_0 \le c_0$ tiles and reuse each B_m for both upper- and lower-triangular contributions.

Single-graph capture. Because the apply path performs no Python branching and no dynamic allocation—all workspace is preallocated at solve setup—the inner PCG loop (one CSR SpMV for $A\mathbf{r}$, one apply, plus the PCG scalar updates) records cleanly into a single `torch.cuda.graph`. Subsequent iterations replay the graph with one `cudaGraphLaunch` call: zero CPU dispatch, zero kernel argument serialization. This captures *the entire* per-iteration solver state. By contrast, IC- and DILU-class preconditioners require sequential triangular solves whose wavefront scheduling creates data hazards across SMs, and AMG V-cycles demand level-by-level synchronization barriers that prevent capture of a single fused graph.

Table 1. Per-iteration preconditioner application cost. nnz denotes the nonzero count of the relevant factor; "Sequential" indicates operations with unavoidable data-dependent wavefront scheduling.

Method	FLOPs	Parallelism	Graph-cap.?
Jacobi	$O(N)$	embarrassing	✓
IC (triangular solve)	$O(\text{nnz})$	sequential	×
Neural SPAI [Yang et al. 2025]	$O(\text{nnz})$	SpMV	✓
AMG (V-cycle)	$O(N \log N)$	level-sync	×
Ours	$O(NL_s)$	batched GEMM	✓

Comparison. Two comparisons inform the design (Table 1). Against IC, our apply is FLOPs-comparable but has no sequential dependency: every GEMM in every stage is issued as a single batched call. Against neural SPAI [Yang et al. 2025], both methods avoid triangular solves, but SPAI apply is dominated by SpMV with irregular memory access (one random gather per nonzero of A) and is therefore bandwidth-bound; our apply consists of batched GEMMs over contiguous tensor blocks and is compute-bound on Tensor Core hardware. Furthermore, SPAI is constrained to the sparsity pattern of A, excluding non-adjacent couplings; our off-diagonal $\mathcal{H}$-tiles represent them explicitly.

6 Training

6.1 Cosine Hutchinson Probe Loss

We train LeafOnlyNet self-supervised. Given probe vectors $\mathbf{Z} \in \mathbb{R}^{N\times K_z}$, we compute $A\mathbf{Z}$ via SpMV (treated as a fixed input, with no gradient through A), apply the preconditioner to get $\hat{\mathbf{Z}} = M(A\mathbf{Z})$, and minimize

$$\mathcal{L}_{\cos} \;=\; 1 \;-\; \frac{1}{K_z}\sum_{k=1}^{K_z} \frac{\hat{\mathbf{z}}_k \cdot \mathbf{z}_k}{\|\hat{\mathbf{z}}_k\|_2\,\|\mathbf{z}_k\|_2}. \tag{7}$$

A perfect preconditioner gives $\mathcal{L}_{\cos} = 0$; the worst case is $\mathcal{L}_{\cos} = 2$ (anti-aligned).

Why cosine. $\mathcal{L}_{\cos}$ is invariant under arbitrary positive rescaling of M: scaling M by $\alpha > 0$ leaves the angle of $\hat{\mathbf{z}}_k$ unchanged. This matches the scale-invariance of PCG convergence, which depends only on the relative spread of eigenvalues of MA, not their absolute location. Frobenius-style objectives—including the SAI loss $\left\| \frac{1}{\|A\|} AM - I \right\|_F^2$ used by [Yang et al. 2025]—instead penalize deviations from $MA = I$ pointwise, implicitly demanding that the eigenvalues of MA cluster near 1, even though PCG converges as fast on any tight cluster. $\mathcal{L}_{\cos}$ removes that location prior, freeing the network to cluster eigenvalues wherever the current preconditioner makes alignment easiest—a behavior we observe directly in Section 7.3 (Fig. 3) and ablate against SAI and an ℓ_2 Hutchinson loss in Section 7.4.2.

6.2 Spectrally Biased Probe Vectors

Naive Gaussian probes weight all spatial frequencies uniformly by energy, so the gradient signal is dominated by high-frequency content—which the diagonal blocks (with full rank L_s per block) already cover. The off-diagonal factors, which encode low-frequency long-range structure with shared rank L_s across $S \gg L_s$ nodes, receive comparatively weak gradients and saturate much later. To

balance the two streams, we apply a small number of damped-Jacobi smoothing steps to the probes before use:

$$\mathbf{z}^{(t+1)} = \mathbf{z}^{(t)} - \omega\, D^{-1} A\, \mathbf{z}^{(t)}, \qquad D = \mathrm{diag}(A), \tag{8}$$

with fixed $\omega = 0.6$ and 2 steps. Each step damps high-frequency components more strongly than low-frequency ones, shifting probe energy toward lower spatial frequencies and delivering proportionally larger gradients to the off-diagonal parameters. The probes are treated as detached inputs, so gradients do not flow back through the smoothing.

6.3 Training Setup

We train with AdamW under a reduce-on-plateau schedule and global gradient clipping. Training contexts (graph, A, masks, padded sizes, smoothed probes) are precomputed once and cached on disk; at each step a mini-batch is drawn at random and padded to a common node count. We set the number of probe vectors to $K_z = \max(64, \lceil\sqrt{N}\rceil)$, balancing gradient noise against compute. The model is compiled with `torch.compile`. All model weights are float32 except the PCG scalar accumulators, which use float64 (Section 7.6).

7 Experiments

7.1 Setup

Hardware. All experiments run on a single [GPU MODEL], CUDA [VERSION], PyTorch [VERSION]. PCG scalar accumulators use float64; preconditioner weights and apply use float32. Inference is timed after one warmup pass with `torch.cuda.synchronize` bracketing; PCG timing uses single-graph CUDA Graph replay (Section 5).

Benchmarks. Our primary benchmark is a stiff multiphase pressure-Poisson system from a particle-based fluid simulator with non-uniform density (contrast ratios up to $\rho = [\,\cdot\,]$), nodes ordered by Morton (Z-curve) reordering. We report scales $N \in \{1024, 2048, 4096, 8192\}$. We hold out $[\cdot]$ frames in-distribution and reserve separate splits at scales and contrast ratios outside the training range.

Baselines. We compare against unpreconditioned CG; Jacobi; IC via `ilupp` (CPU) and AMGX `MULTICOLOR_DILU` (GPU) [Naumov et al. 2015]; PyAMG (CPU) and AMGX classical AMG (GPU); and Neural SPAI [Yang et al. 2025] retrained on our data. All GPU baselines use CUDA Graph replay where their algorithm permits it; IC and AMG do not, since their sequential triangular solves and V-cycle level synchronization prevent single-graph capture.

Best configuration. Unless noted, results use $d = [\cdot]$, $n_l = [\cdot]$, $L = 128$, $p_{\mathrm{diag}} = 1$, $p_{\mathrm{off}} = 4$, $n_{\mathrm{gcn}} = 2$, highways enabled.

7.2 Main Performance

Table 2 summarizes the headline result. At the largest scale we evaluate, $N = 8\,192$, our preconditioner reduces the PCG iteration count from 2 163 (unpreconditioned) to 141, with an end-to-end wall-clock cost of 15.7 ms (4.0 ms inference + 11.7 ms PCG): 3.3× faster than GPU Jacobi, 18.1× faster than GPU AMG (AMGX), and 31.4× faster than GPU IC-class (AMGX MULTICOLOR_DILU). The classical methods either spend their budget on triangular-solve serialization (DILU, 390.8 ms solve) or on per-iteration AMG V-cycle work (174.0 ms solve) despite their much smaller iteration counts. Total time grows from 5.7 ms at $N = 1\,024$ to 15.7 ms at $N = 8\,192$—a 2.7× increase for an 8× increase in problem size, consistent with the $\Theta(N)$ scaling argument and reflecting the relatively flat inference cost across scales.

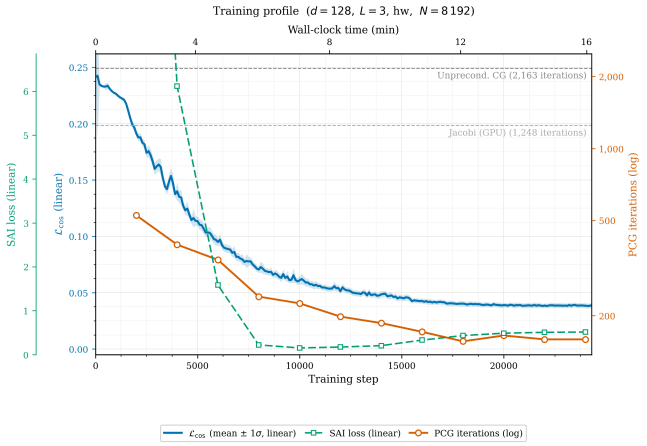

Fig. 3. Training profile of the configuration used in Table 2 ($N = 8\,192$, $d = 128$, $n_l = 3$, highways). Blue: $\mathcal{L}_{\cos}$ on the training set (mean $\pm 1\sigma$, last 100 steps). Green dashed: SAI loss [Yang et al. 2025] computed on a held-out validation frame at the same checkpoints (*not* used for training). Orange: PCG iteration count to rtol $= 10^{-8}$ on that frame, log scale. Top axis: wall-clock minutes. Horizontal grey lines: unpreconditioned CG and Jacobi GPU baselines.

7.3 Training Dynamics

Figure 3 provides our clearest empirical evidence for the design rationale of $\mathcal{L}_{\cos}$. Total wall-clock training time on a single GPU is 16.2 min for 24 300 optimizer steps; the model beats GPU Jacobi by step ∼2,000 (∼1.3 min) and drops under 200 PCG iterations by step ∼12,000 (∼8 min). The structural finding is in the relationship *between* the three curves. Up to step ∼ 8 000, all three quantities decrease together. After that, the SAI loss plateaus and then *rises*, from ∼ 10^{-3} at step 10 000 to ∼ 0.5 by step 24 000, even as $\mathcal{L}_{\cos}$ continues to decrease smoothly and PCG iteration count continues to fall in lockstep with it. The model is moving the eigenvalues of MA *away* from $\lambda = 1$ in pointwise terms, but *toward* a tighter relative cluster wherever that turns out to be—the exact behavior we predicted analytically in Section 6.1, and the single strongest argument for using a scale- and location-invariant cosine objective rather than a Frobenius surrogate.

7.4 Ablations

We ablate the architecture and the loss to isolate the contribution of each component, all trained for $[\cdot]$ steps at $N = 4\,096$ for control.

7.4.1 Architecture. The component ablations remove each design element in turn from the best configuration.

Table 2. PCG solve performance across problem scales on the multiphase Poisson benchmark (relative residual tolerance 10^{-8}). For each scale and method we report Total time (setup + solve, ms) and Iters. "†" = CUDA Graph not supported by the algorithm; times include Python dispatch. Best GPU total time per scale in **bold**.

	$N = 1{,}024$		$N = 2{,}048$		$N = 4{,}096$		$N = 8{,}192$	
Method	Total	Iters	Total	Iters	Total	Iters	Total	Iters
Unpreconditioned CG (GPU)	TODO	TODO	TODO	TODO	TODO	TODO	88.2	2163
Jacobi (GPU)	8.0	TODO	13.0	TODO	21.0	TODO	51.2	1248
IC / DILU (GPU)†	TODO	TODO	TODO	TODO	TODO	TODO	492.4	32
AMG / AMGX (GPU)†	TODO	TODO	TODO	TODO	TODO	TODO	284.3	2
Neural SPAI [Yang et al. 2025]	TODO	TODO	TODO	TODO	TODO	TODO	TODO	TODO
Ours (GPU)	**5.7**	TODO	**5.7**	TODO	**7.8**	TODO	**15.7**	141
Ours, breakdown:								
Inference	1.8		1.8		2.1		4.0	
PCG (CUDA Graph)	3.9		3.9		5.7		11.7	

Table 3. Architecture and loss ablations at $N = 4\,096$. "Inf." = inference (ms); "Solve" = PCG solve (ms); "Iters" = PCG iterations to rtol = 10^{-8}; all wall-clock with CUDA Graph replay. Best per column in **bold**.

Configuration	Inf.	Solve	Iters	Params
Capacity				
d=64, n_l=2	TODO	TODO	TODO	TODO
d=128, n_l=2	TODO	TODO	TODO	TODO
d=128, n_l=3	TODO	TODO	TODO	TODO
d=256, n_l=2	TODO	TODO	TODO	TODO
Components (best − component)				
− highways	TODO	TODO	TODO	TODO
− off-diag stack	TODO	TODO	TODO	TODO
− GCN (n_{gcn}=0)	TODO	TODO	TODO	TODO
− Jacobi residual gate	TODO	TODO	TODO	TODO
− probe smoothing	TODO	TODO	TODO	TODO
Loss (best architecture)				
$\mathcal{L}_{\ell_2}$ Hutchinson	TODO	TODO	TODO	–
SAI [Yang et al. 2025]	TODO	TODO	TODO	–
$\mathcal{L}_{\cos}$ (ours)	**TODO**	**TODO**	**TODO**	–
Best (full)	**TODO**	**TODO**	**TODO**	**TODO**

Table 4. Generalization to out-of-distribution problem scales, contrast ratios, and problem types. Iterations to rtol = 10^{-8}. "†" = did not converge in $[\cdot]$ iterations.

Split	Ours	Neural SPAI	IC	AMG
ID scale, ID ρ	TODO	TODO	TODO	TODO
OOD scale (smaller)	TODO	TODO	TODO	TODO
OOD scale (larger)	TODO	TODO	TODO	TODO
OOD contrast (high)	TODO	TODO	TODO	TODO
OOD problem type	TODO	TODO	TODO	TODO

7.4.2 Loss. We compare $\mathcal{L}_{\cos}$ against an ℓ_2 Hutchinson loss $\mathcal{L}_{\ell_2} = \mathbb{E}_{\mathbf{z}}[\|MA\mathbf{z} - \mathbf{z}\|_2^2]$ and the SAI loss of [Yang et al. 2025], all on the same architecture and probes. The training-curve divergence of $\mathcal{L}_{\cos}$ and SAI in Fig. 3 sets the expectation for this comparison: a model trained against SAI directly should plateau in PCG iterations as soon as the cluster reaches $\lambda = 1$, while a model trained against $\mathcal{L}_{\cos}$ should continue to tighten the cluster wherever it is. We report PCG iteration count, the achieved condition number $\kappa(MA)$, and the cluster location median$(\lambda(MA))$ for each loss.

7.5 Generalization

7.6 Precision and Hardware Efficiency

8 Limitations and Future Work

Three limitations of our current design are worth flagging. First, the $\mathcal{H}$-matrix partition is fixed by the index ordering of A; problems whose discretization has poor spatial locality after Morton/RCM (e.g. extremely irregular unstructured meshes with mixed element types) may exhibit weaker low-rank structure on the admissible tiles, making our compression less effective. Second, the partition is keyed to N and to the choice of L: dramatic changes in problem scale change K and $M_{\mathcal{H}}$, and in our current implementation the model is trained for a specific $\mathcal{H}$-matrix layout. Generalization across small scale changes (Section 7.5) is encouraging, but extreme rescaling (e.g. 4× larger problems) currently requires retraining at the new layout. Third, our preconditioner is symmetric by construction but not strictly SPD: the diagonal contribution $\sum_k P_k F_k F_k^\top P_k^\top$ is positive semidefinite, but the off-diagonal contributions are symmetric and generally indefinite. PCG nevertheless converges robustly in our experiments, likely because training drives MA's spectrum into a numerically stable region on the relevant probe directions; flexible GMRES [Saad 1993] would lift this implicit assumption and extend the same preconditioner to non-symmetric systems. We leave that direction, along with input-adaptive partitioning that preserves single-graph capture, to future work.

9 Conclusion

We presented a hierarchical transformer-based neural preconditioner anchored to a fixed weak-admissibility $\mathcal{H}$-matrix partition. Diagonal blocks are emitted at full rank, off-diagonal blocks at a shared coarse rank, yielding a $\Theta(N)$ forward pass that addresses both high-frequency local and low-frequency long-range errors. The apply path is a deterministic sequence of batched GEMMs over

contiguous memory, captured as a single CUDA Graph. The architecture moves neural preconditioning away from sparsity-pattern constraints toward full-graph structured representations whose bias derives from the spectral geometry of the underlying physics.

Acknowledgments

Omitted for anonymous review.

References

Michele Benzi, Carl D. Meyer, and Miroslav Tůma. 1996. A Sparse Approximate Inverse Preconditioner for the Conjugate Gradient Method. *SIAM Journal on Scientific Computing* 17, 5 (1996), 1135–1149.

Steffen Börm, Lars Grasedyck, and Wolfgang Hackbusch. 2003. *Introduction to Hierarchical Matrices with Applications.* Engineering Analysis with Boundary Elements. Survey/textbook treatment of $\mathcal{H}$-matrices and $\mathcal{H}^2$-matrices..

William L. Briggs, Van Emden Henson, and Steve F. McCormick. 2000. *A Multigrid Tutorial* (2 ed.). SIAM, Philadelphia, PA.

Jie Chen. 2024. Graph Neural Preconditioners for Iterative Solutions of Sparse Linear Systems. *arXiv preprint arXiv:2406.00809* (2024). https://arxiv.org/abs/2406.00809 Published at ICLR 2025; see also Chen2025graph entry in this file..

Yuwei Fan, Lin Lin, Lexing Ying, and Leonardo Zepeda-Núñez. 2019. A Multiscale Neural Network Based on Hierarchical Matrices. *arXiv preprint arXiv:1807.01883* (2019). https://arxiv.org/abs/1807.01883

Emilio McAllister Fognini, Marta M. Betcke, and Ben T. Cox. 2025. Learning Green's Operators through Hierarchical Neural Networks Inspired by the Fast Multipole Method. *arXiv preprint arXiv:2509.20591* (2025). https://arxiv.org/abs/2509.20591

Wolfgang Hackbusch. 1999. *A Sparse Matrix Arithmetic Based on $\mathcal{H}$-Matrices. Part I: Introduction to $\mathcal{H}$-Matrices.* Springer, Berlin. Computing 62(2):89–108. Often cited as hierarchical matrix foundations..

Wolfgang Hackbusch and Boris N. Khoromskij. 2002. *Adaptive $\mathcal{H}$-Matrix Approximation on General Domains.* Springer. Survey of adaptive hierarchical matrix techniques..

Tucker Hartland, Georg Stadler, Mauro Perego, Kim Liegeois, and Noémi Petra. 2023. Hierarchical Off-Diagonal Low-Rank Approximation of Hessians in Inverse Problems, with Application to Ice Sheet Model Initialization. *arXiv preprint arXiv:2301.03644* (2023). https://arxiv.org/abs/2301.03644

Paul Häusner, Ozan Öktem, and Jens Sjölund. 2023. Neural Incomplete Factorization: Learning Preconditioners for the Conjugate Gradient Method. *arXiv preprint arXiv:2305.16368* (2023). https://arxiv.org/abs/2305.16368

Michael F. Hutchinson. 1989. A Stochastic Estimator of the Trace of the Influence Matrix for Laplacian Smoothing Splines. *Communications in Statistics—Simulation and Computation* 18, 3 (1989), 1059–1076.

Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR).*

L. Yu. Kolotilina and A. Yu. Yeremin. 1993. Factorized Sparse Approximate Inverse Preconditionings I: Theory. *SIAM J. Matrix Anal. Appl.* 14, 1 (1993), 45–58.

Yichen Li, Peter Yichen Chen, Tao Du, and Wojciech Matusik. 2023. Learning Preconditioners for Conjugate Gradient PDE Solvers. *arXiv preprint arXiv:2305.16432* (2023). https://arxiv.org/abs/2305.16432 Public drafts also used the name Neural PCG (ICLR 2023)..

Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. 2020a. Multipole Graph Neural Operator for Parametric Partial Differential Equations. *Advances in Neural Information Processing Systems* 33 (2020).

Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. 2020b. Neural Operator: Graph Kernel Network for Partial Differential Equations. *arXiv preprint arXiv:2003.03485* (2020). https://arxiv.org/abs/2003.03485

Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. 2021. Fourier Neural Operator for Parametric Partial Differential Equations. In *International Conference on Learning Representations.* https://openreview.net/forum?id=c8P9NQVtmnO

Chih-Jen Lin and Jorge J. Moré. 1999. Incomplete Cholesky Factorizations with Limited Memory. *SIAM Journal on Scientific Computing* 21, 1 (1999), 24–45.

Lijun Liu, Shengguo Li, Xiangke Hu, Yutong Wang, Xuejun Liu, and Jingling Xue. 2016. Exploring Data Level Parallelism in Incomplete LU Factorization on GPUs. *IEEE Transactions on Parallel and Distributed Systems* 27, 12 (2016), 3397–3410.

Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, and George Em Karniadakis. 2021. Learning Nonlinear Operators via DeepONet Based on the Universal Approximation Theorem of Operators. *Nature Machine Intelligence* 3, 3 (2021), 218–229.

Ilay Luz, Meirav Galun, Haggai Maron, Ronen Basri, and Irad Yavneh. 2020. Learning Algebraic Multigrid Using Graph Neural Networks. *arXiv preprint arXiv:2003.05744* (2020). https://arxiv.org/abs/2003.05744

Kangbo Lyu, Ruihong Cen, Yushen Wu, and Tao Du. 2026. A Multigrid-Inspired Neural Iterative Solver for Poisson Equations on Large Voxel Grids. https://openreview.net/forum?id=lNcbGSWhJo. OpenReview workshop/conference submission..

Maxim Naumov, Michael Chien, Paul Vandermersch, Ujval Kapasi, Boris Catanzaro, and Michael Garland. 2015. cuSPARSE Library. In *GPU Technology Conference (GTC).* NVIDIA AMGX / sparse linear algebra stack; widely referenced for GPU AMG and sparse solvers..

Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. 2019. Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations. *J. Comput. Phys.* 378 (2019), 686–707.

Kirill Rudikov, Anastasia Markeeva, Vasily Bulatov, and Dmitry Vetrov. 2024. Neural Functional Operator for Parametric PDEs. *arXiv preprint arXiv:2402.01030* (2024). https://arxiv.org/abs/2402.01030

John W. Ruge and Klaus Stüben. 1987. Algebraic Multigrid (AMG). *Multigrid Methods* 3 (1987), 73–130.

Yousef Saad. 1993. A Flexible Inner-Outer Preconditioned GMRES Algorithm. *SIAM Journal on Scientific Computing* 14, 2 (1993), 461–469.

Yousef Saad. 2003. *Iterative Methods for Sparse Linear Systems* (2 ed.). SIAM, Philadelphia, PA.

Pietro Sittoni, Emanuele Zangrando, Angelo Alberto Casulli, Nicola Guglielmi, and Francesco Tudisco. 2026. Neural-HSS: Hierarchical Semi-Separable Neural PDE Solver. *arXiv preprint arXiv:2602.18248* (2026). https://arxiv.org/abs/2602.18248

Konstantin Trifonov et al. 2025. Graph Neural Networks for Preconditioning Linear Systems. *arXiv preprint* (2025). Placeholder entry; replace with definitive citation when available..

Konstantin Trifonov and Stefan Güttel. 2024. Linear Algebraic Preconditioning using Machine Learning. *arXiv preprint arXiv:2403.11463* (2024). https://arxiv.org/abs/2403.11463 Verify final venue if published..

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems 30 (NIPS 2017).* 5998–6008.

Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. 2019. Simplifying Graph Convolutional Networks. In *Proceedings of the 36th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 97).* PMLR, 6861–6871. http://proceedings.mlr.press/v97/wu19e.html

Tianshi Xu, Rui Peng Li, and Yuanzhe Xi. 2025. Neural Approximate Inverse Preconditioners. *arXiv preprint arXiv:2510.13034* (2025). https://arxiv.org/abs/2510.13034

Ichitaro Yamazaki, Stanimire Tomov, and Jack Dongarra. 2020. Mixed-Precision Cholesky QR Factorization and Its Parallelization on GPUs. *Parallel Comput.* 99 (2020), 102693.

Zherui Yang, Zhehao Li, Kangbo Lyu, Yixuan Li, Tao Du, and Ligang Liu. 2025. Learning Sparse Approximate Inverse Preconditioners for Conjugate Gradient Solvers on GPUs. *arXiv preprint arXiv:2510.27517* (2025). https://arxiv.org/abs/2510.27517

Received 12 May 2026; revised ; accepted